

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет Программной Инженерии и Компьютерной Техники

Направление подготовки (специальность) Проектирование и разработка систем больших
данных(09.04.04 Программная инженерия)

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
по дисциплине «Структуры и алгоритмы в базах данных и распределенных
системах»

Тема лабораторной работы: **Хэширование**

Обучающийся: Зарубин Владислав Александрович
(Фамилия И.О.)

P4135
(номер группы)

Преподаватель: Платонов Алексей Владимирович
(Фамилия И.О.)

Санкт-Петербург

2026 г.

Задание

Необходимо реализовать три алгоритма (в скобках операции):

- 1) хэш таблицу на файловой системе с бакетами (вставка, обновление и удаление)
- 2) perfect hash (создание полного индекса по фиксированному набору ключей и поиск)
- 3) lsh для поиска дублей в текстах или точек в 3d (создание индекса, добавление новых точек, фулл скан для поиска дублей)

По всем алгоритмам должны быть:

- 1) тесты функциональности на случайных наборах данных (набор данных при тесте генерируется) - очень помогает ловить корнер-кейсы
- 2) перфтесты на больших объемах данных - считаем пропускную способность и время операции
- 3) профайлинг памяти и цпу всех реализаций с анализом узких мест и набором гипотез как можно сделать лучше

Ход работы

Аппаратные характеристики:

- **операционная система:** macOS Sequoia 15.6 (arm64);
- **процессор:** Apple M4 (10 ядер)
- **оперативная память:** 24 ГБ
- **версия Java:** 17.0.18

Реализация

PerfectHash

В работе реализован статический двухуровневый perfect hash для набора уникальных строковых ключей. На этапе построения все ключи сначала переводятся в числовой вид, после чего для них подбирается первичная хеш-функция, которая распределяет элементы по корзинам без слишком большого числа коллизий. Затем для каждой корзины строится своя вторичная таблица, в которой уже подбирается хеш без коллизий. В результате после построения индекс поддерживает быстрый поиск ключа по фиксированному набору данных.

FileHashTable

Файловая hash table реализована на основе extensible hashing. Таблица хранит директорию бакетов и сами бакеты в отдельных файлах, а доступ к ним выполняется через memory-mapped files. При вставке ключ по хешу попадает в бакет; если бакет переполнен, выполняется split и при необходимости расширяется директория. Реализация поддерживает вставку, поиск, обновление и удаление записей, а основная идея состоит в том, чтобы динамически расширять структуру без полной перестройки всей таблицы.

LSH

LSH реализован для поиска дубликатов среди трехмерных точек. Для каждой точки строится битовая сигнатура по набору случайных гиперплоскостей, после чего точки с одинаковой сигнатурой попадают в один bucket. Поиск дубликатов выполняется либо через просмотр кандидатов внутри этих bucket-ов, либо через полный перебор для сравнения.

Методика замеров:

Для измерения производительности реализованных структур использовался JMH. Во всех бенчмарках применялись одинаковые параметры запуска: 5 warmup-итераций, 20 measurement-итераций по 1 секунде и 5 forks, чтобы уменьшить влияние случайных колебаний и получить стабильные результаты. Для FileHashTable отдельно измерялись операции поиска, обновления, удаления и вставки нового элемента. Дополнительно был добавлен бенчмарк поиска по фиксированному hot-set из 64 – а этот эксперимент позволяет отделить алгоритмическую стоимость операции от вклада cache miss penalty при росте размера таблицы. Для PerfectHashIndex отдельно измерялись построение индекса (buildIndex) и поиск, причём для поиска снимались как сценарий попадания (lookupHit), так и промаха (lookupMiss). Для LSH измерялись построение индекса, добавление новых точек, поиск дублей через LSH и поиск дублей полным перебором. Для всех операций, где это уместно, снимались как latency, так и throughput.

Таблица 1 – Результаты бенчмарка задержки в extendible hashing (среднее по итерациям)

N	Вставка, нс/оп	Обновление, нс/оп	Удаление, нс/оп	Поиск, нс/оп
2000	5.104 ± 0.447	0.103 ± 0.001	0.064 ± 0.001	0.063 ± 0.001
4000	7.044 ± 0.611	0.108 ± 0.001	0.071 ± 0.001	0.068 ± 0.001
6000	4.007 ± 0.336	0.114 ± 0.001	0.069 ± 0.001	0.070 ± 0.001
8000	9.092 ± 0.812	0.114 ± 0.001	0.071 ± 0.001	0.073 ± 0.001
10000	5.555 ± 0.487	0.115 ± 0.001	0.072 ± 0.001	0.076 ± 0.001
12000	5.839 ± 0.512	0.118 ± 0.001	0.073 ± 0.001	0.076 ± 0.001

Таблица 2 – Результаты бенчмарка throughput в extendible hashing (среднее по итерациям)

N	Вставка, опс/оп	Обновление, опс/оп	Удаление, опс/оп	Поиск, опс/оп
2000	217,212 ± 16,757	9,527,551 ± 67,429	14,542,037 ± 830,270	15,592,937 ± 121,628
4000	155,147 ± 13,091	9,145,169 ± 58,534	15,044,437 ± 205,504	14,719,500 ± 116,862
6000	275,405 ± 22,778	8,822,883 ± 78,059	14,271,446 ± 255,883	14,204,352 ± 148,297
8000	123,156 ± 11,627	8,766,989 ± 92,942	14,022,076 ± 121,628	13,714,280 ± 155,848
10000	195,166 ± 25,346	8,628,866 ± 86,084	13,533,335 ± 155,944	13,209,532 ± 124,217
12000	198,026 ± 22,127	8,487,421 ± 83,575	13,131,866 ± 244,611	13,072,321 ± 141,325

Get

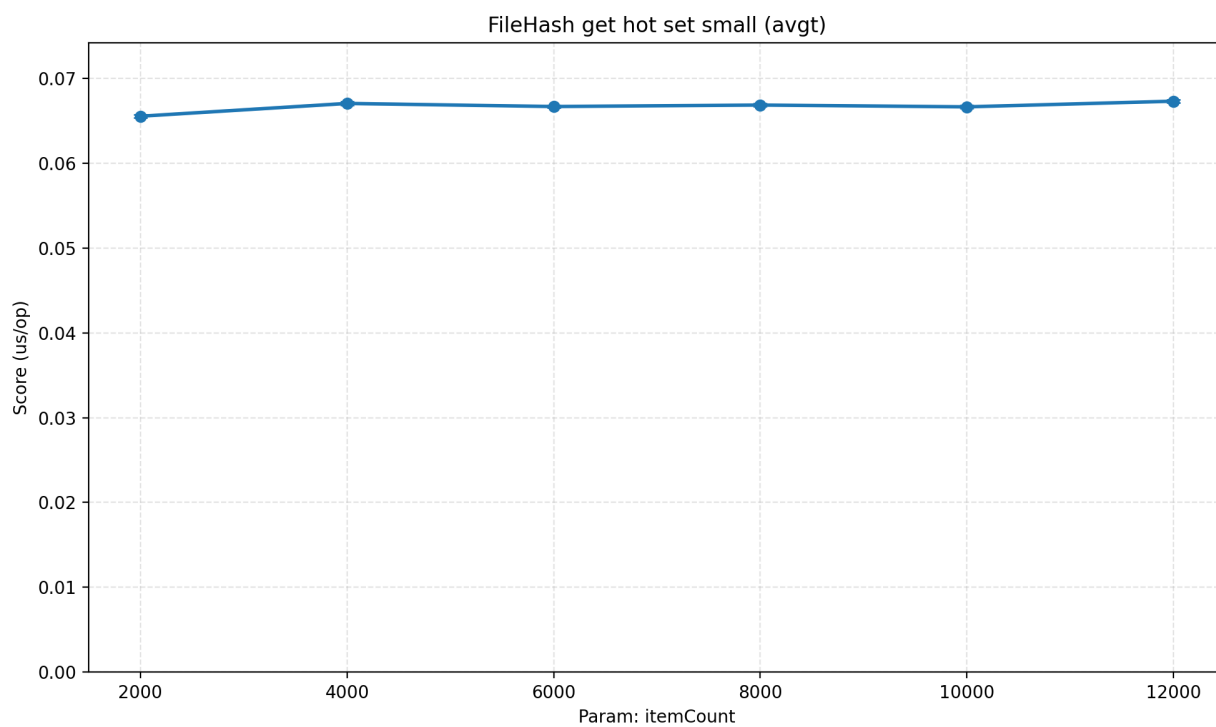


Рисунок 1 – График latency операции Get

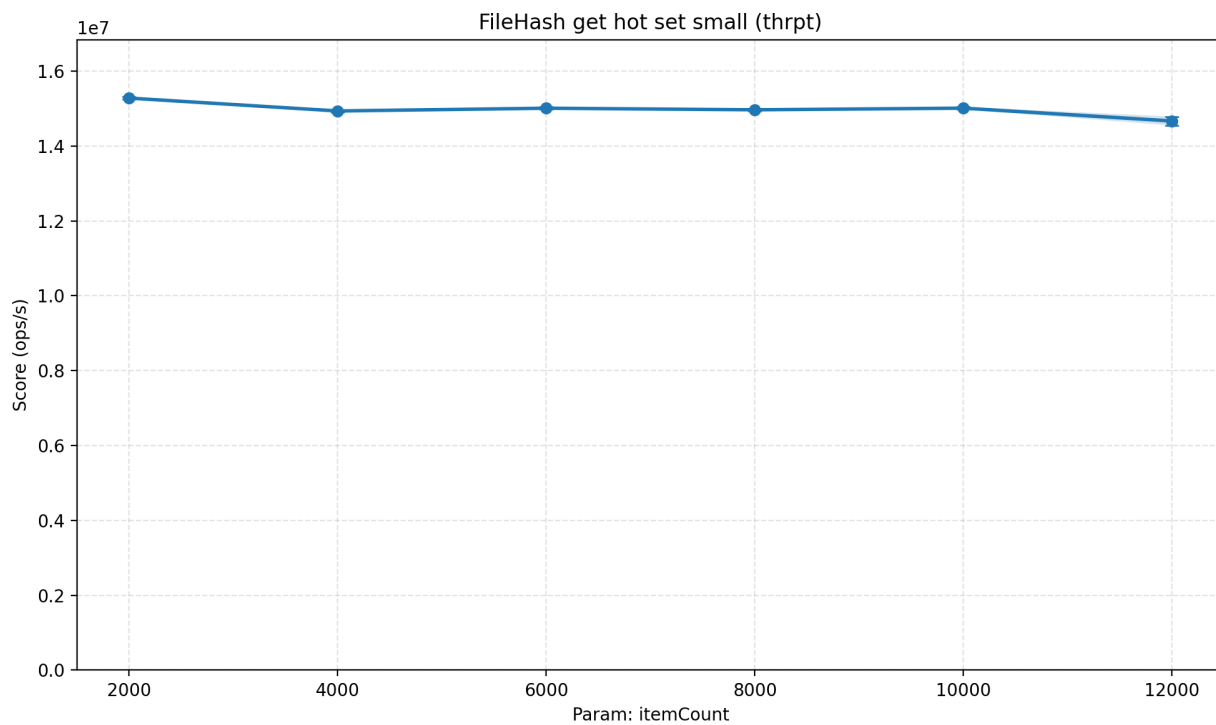


Рисунок 2 – График throughput для операции Get

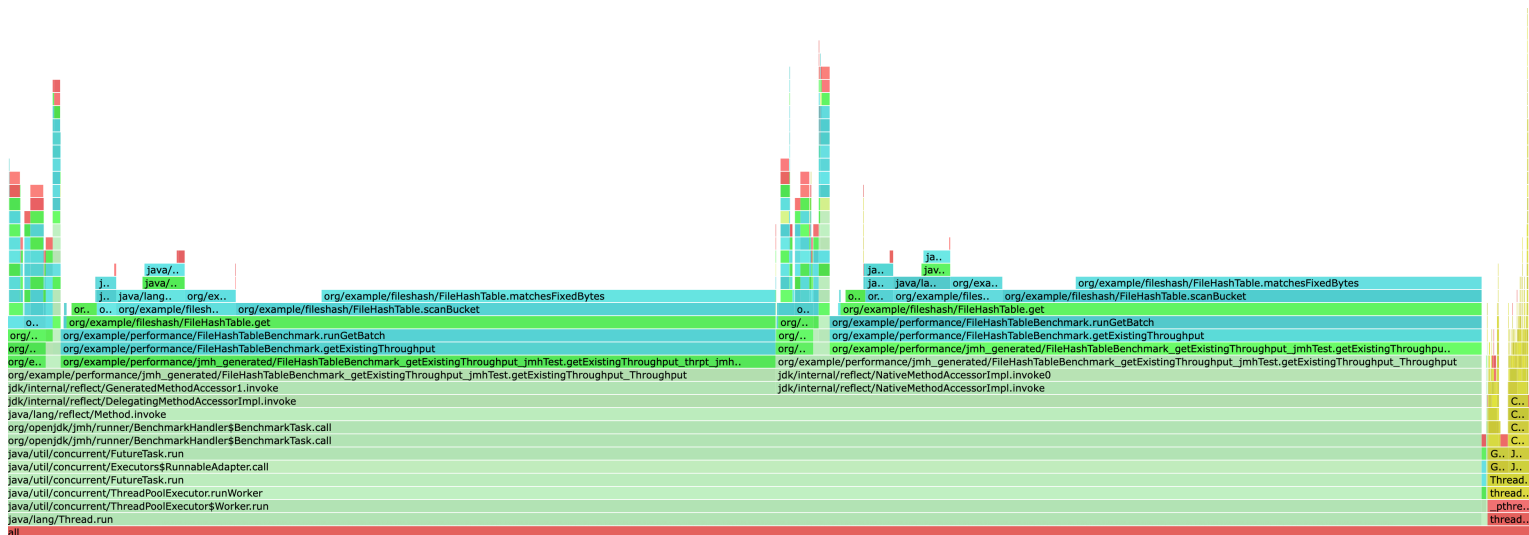


Рисунок 3 – CPU-профиль для операции Get

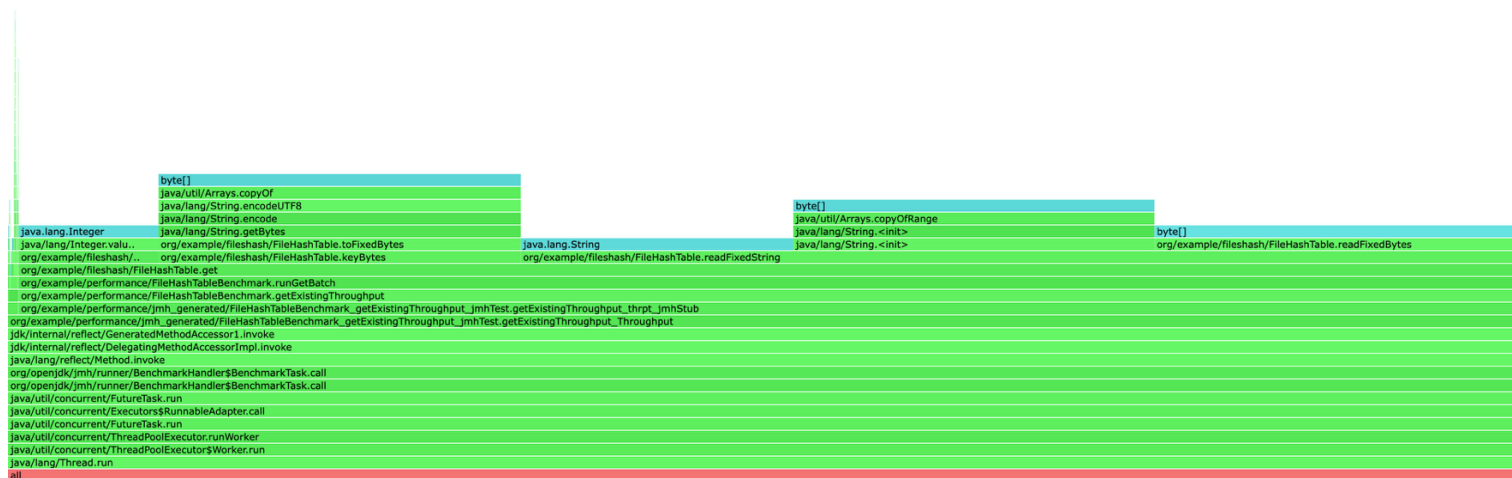


Рисунок 4 – Профиль аллокаций операции Get

Get алгоритмически работает за $O(1)$ — подтверждено экспериментом с фиксированным hot-set: при росте N от 2000 до 12000 latency остаётся плоской. CPU-профиль показывает горячие точки scanBucket и matchesFixedBytes. Allocation profile видит накладные расходы на byte[] в keyBytes и new String в readFixedString. Возможные улучшения: убрать промежуточные byte[] за счёт прямого сравнения по MappedByteBuffer, либо уменьшить bucketCapacity.

Insert

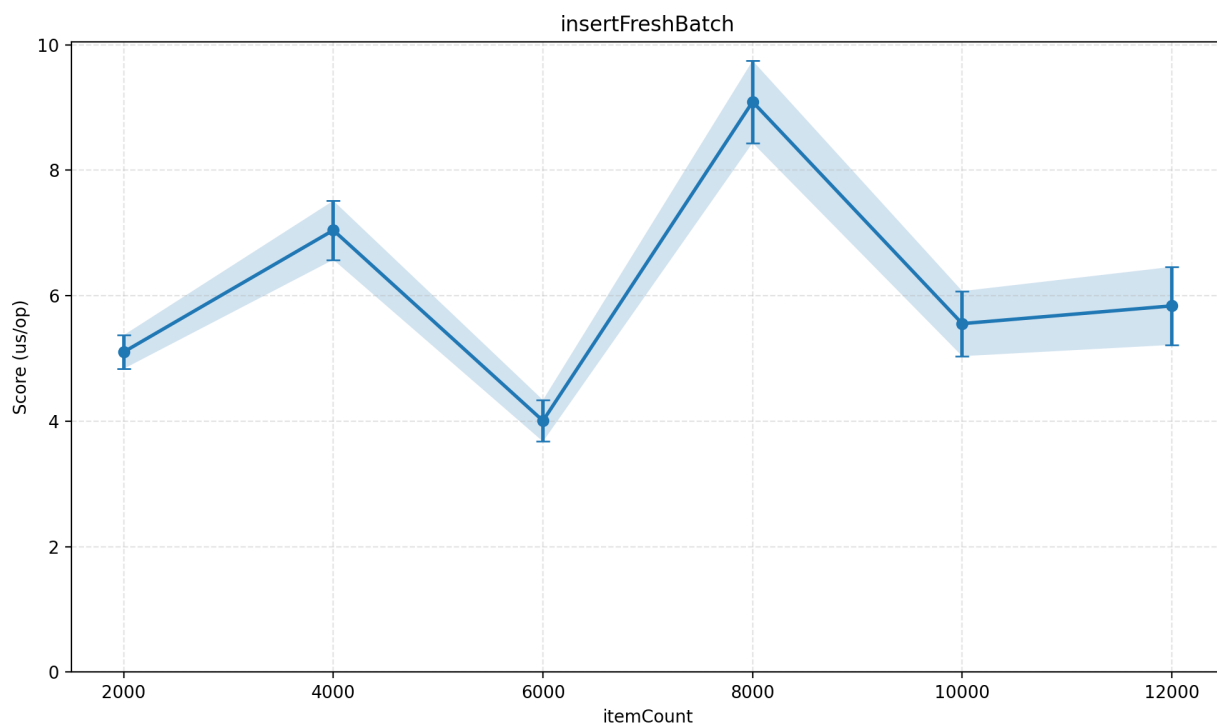


Рисунок 5 – График latency для операции insert

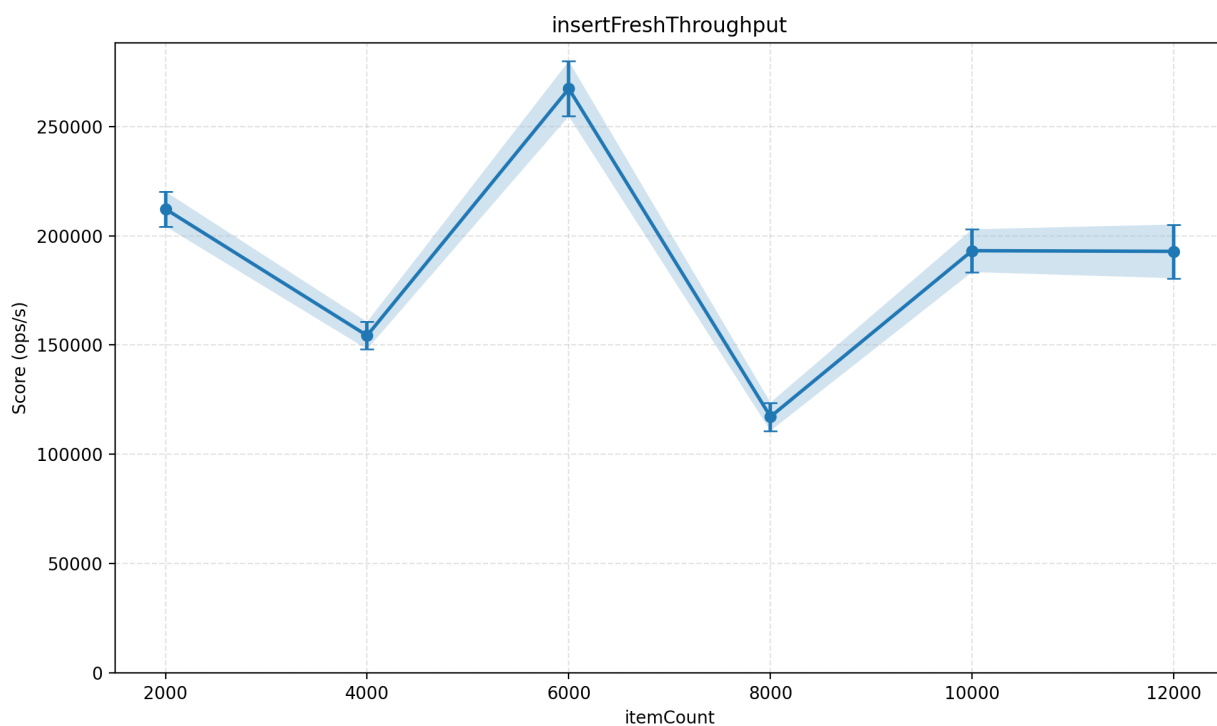


Рисунок 6 – График throughput для insert

ВЫЗОВЫ.

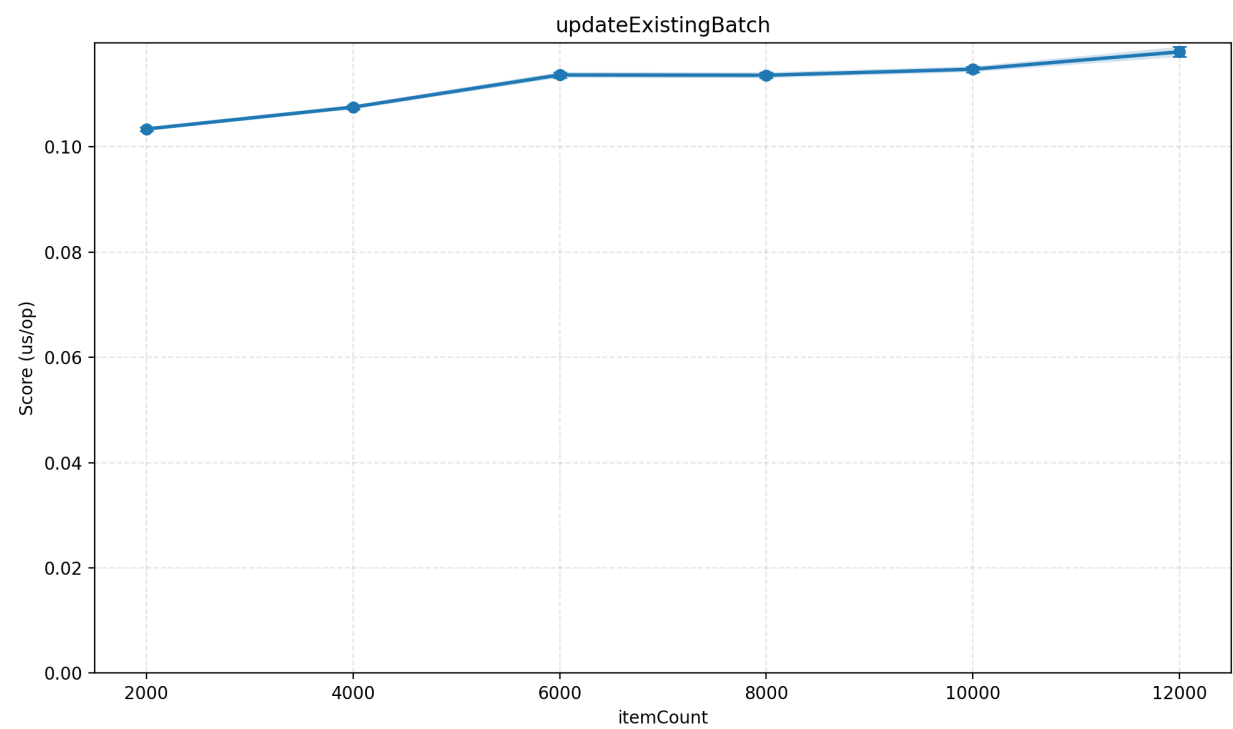


Рисунок 9 – График latency для операции update

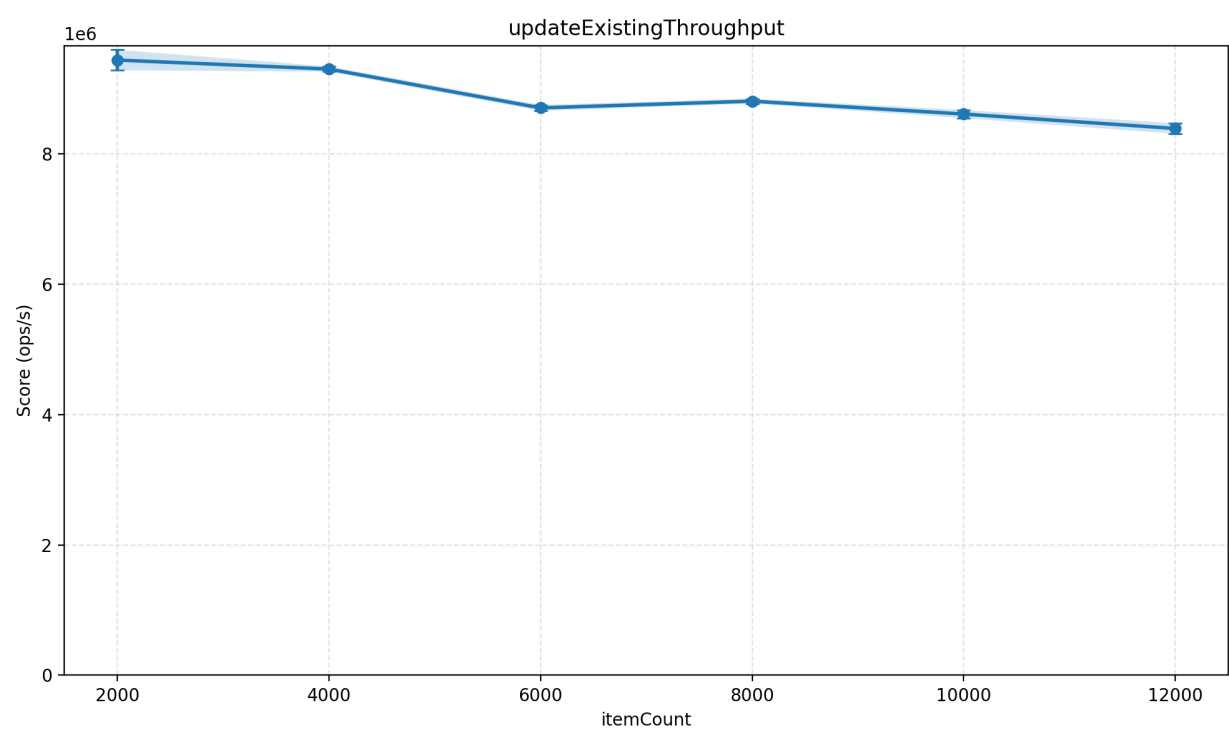


Рисунок 10 – График throughput для операции update

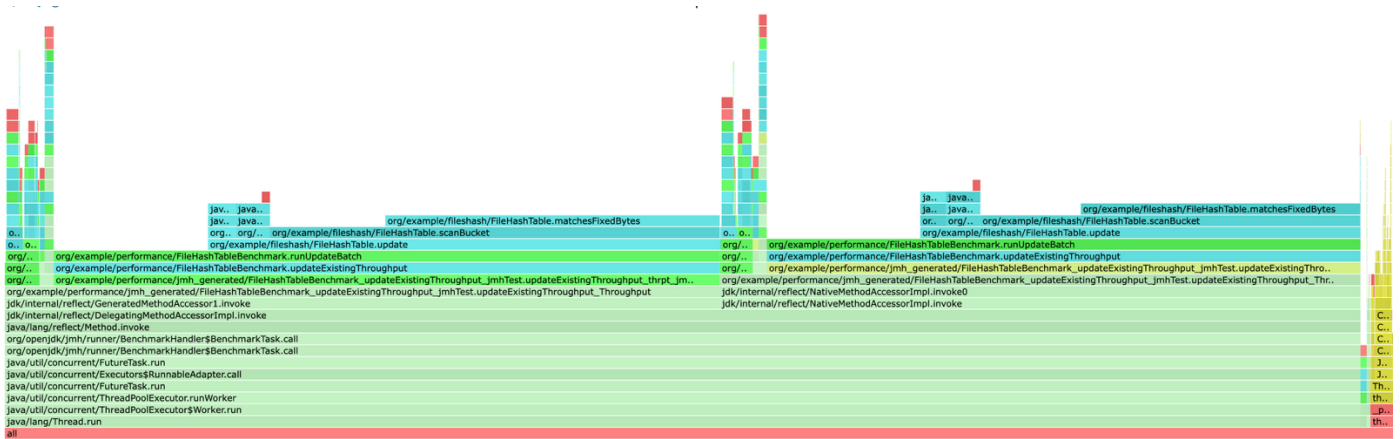


Рисунок 11 – CPU-профиль для операции update

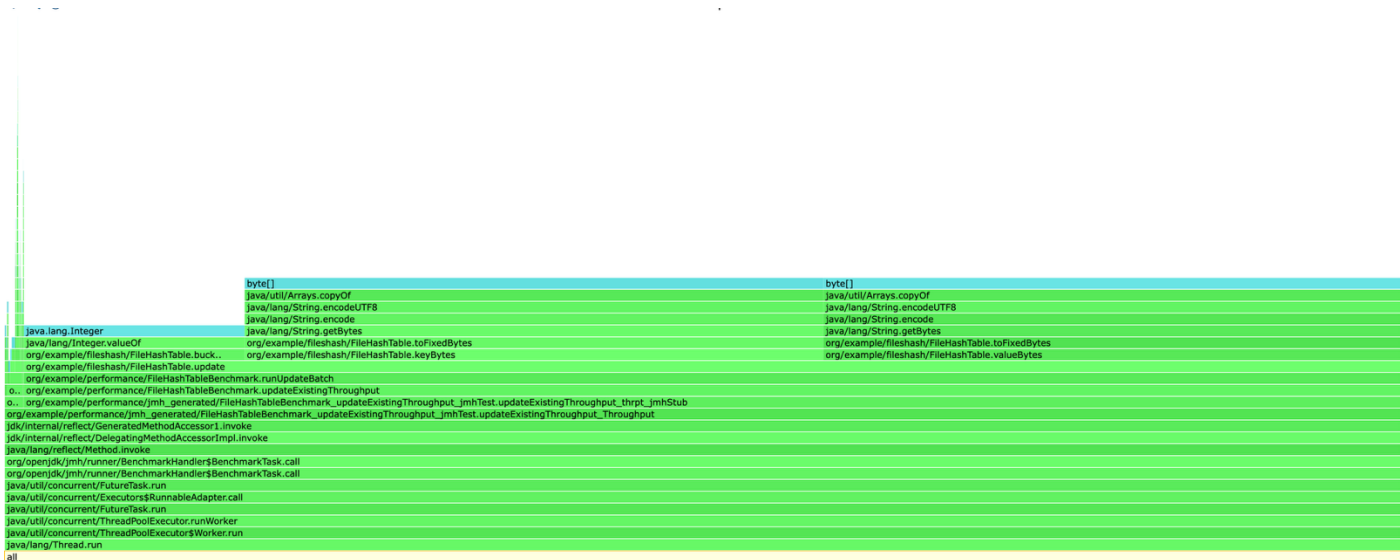


Рисунок 12 – Профиль аллокаций для операции update

Latency операции update постепенно растет с увеличением N. Основное время уходит на scanBucket и сравнение ключей. Профилирование это подтверждает: в CPU profile доминируют scanBucket и matchesFixedBytes. Allocation profile показывает аллокации string и byte, которые возникают при работе с данными. Throughput плавно снижается, так как каждая операция становится немного дороже. В отличие от insert, здесь нет резких скачков, так как update не вызывает split бакетов и не приводит к дорогим операциям перераспределения.

Delete

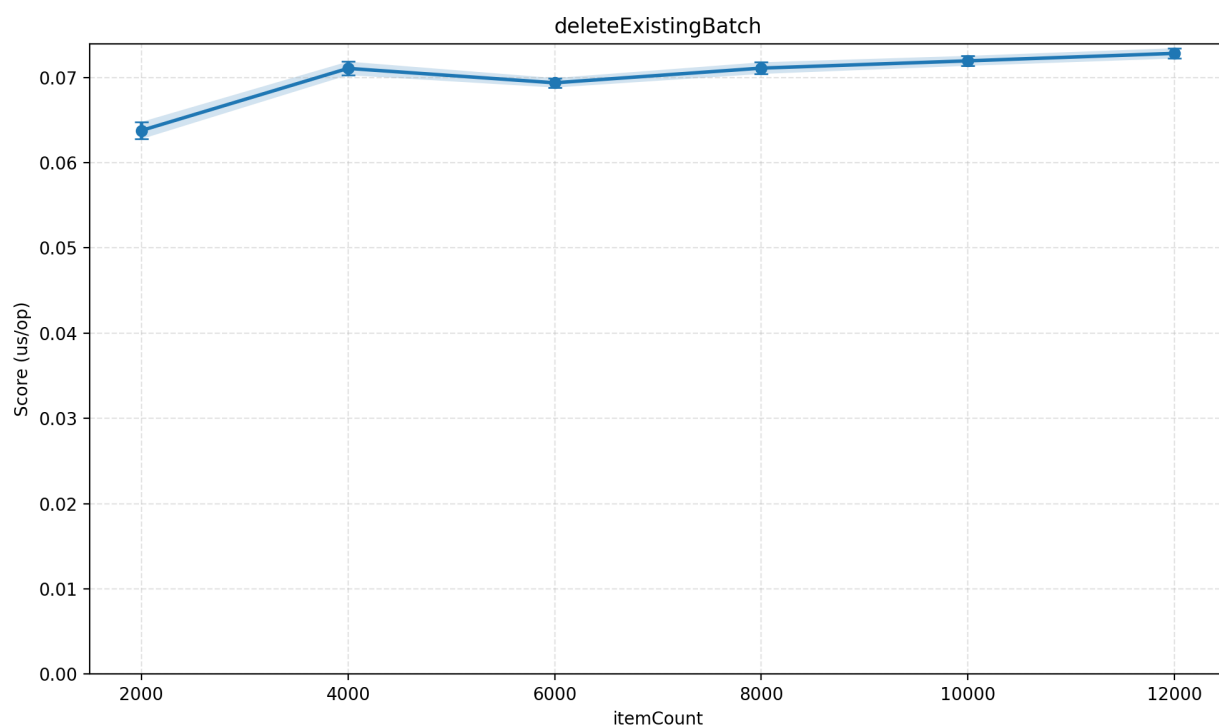


Рисунок 13 – График latency для операции delete

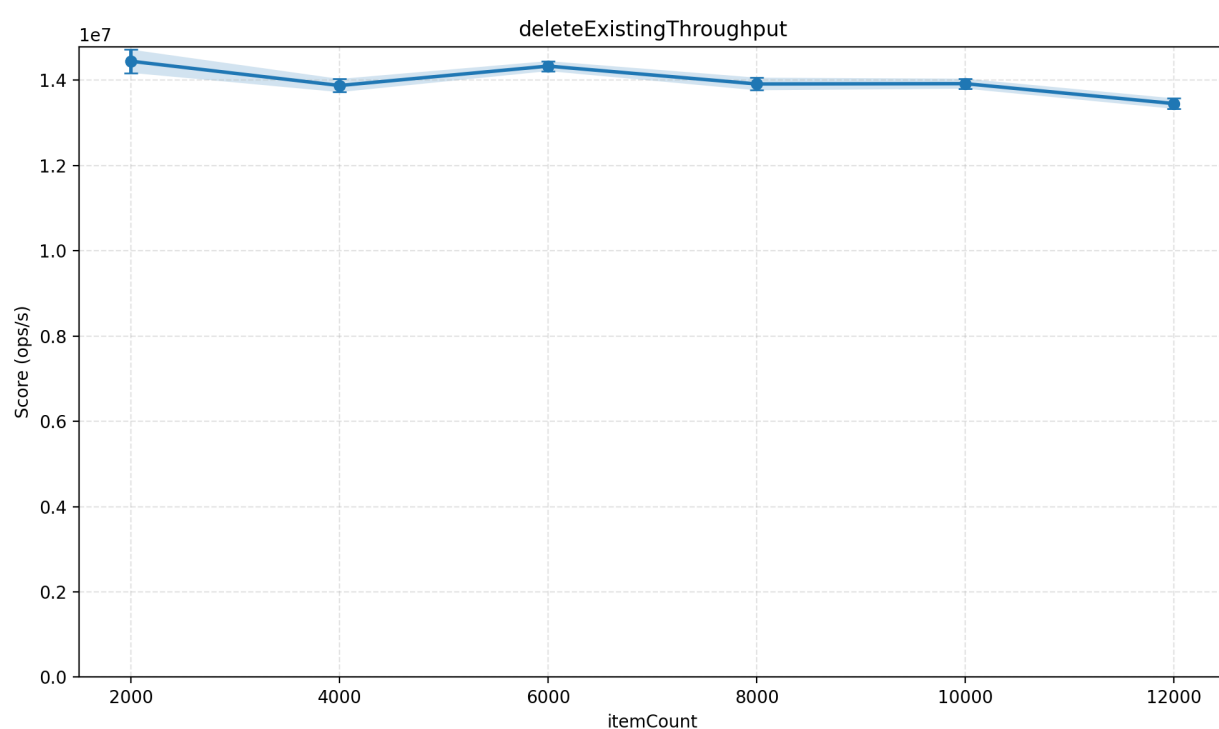


Рисунок 14 – График throughput для операции delete

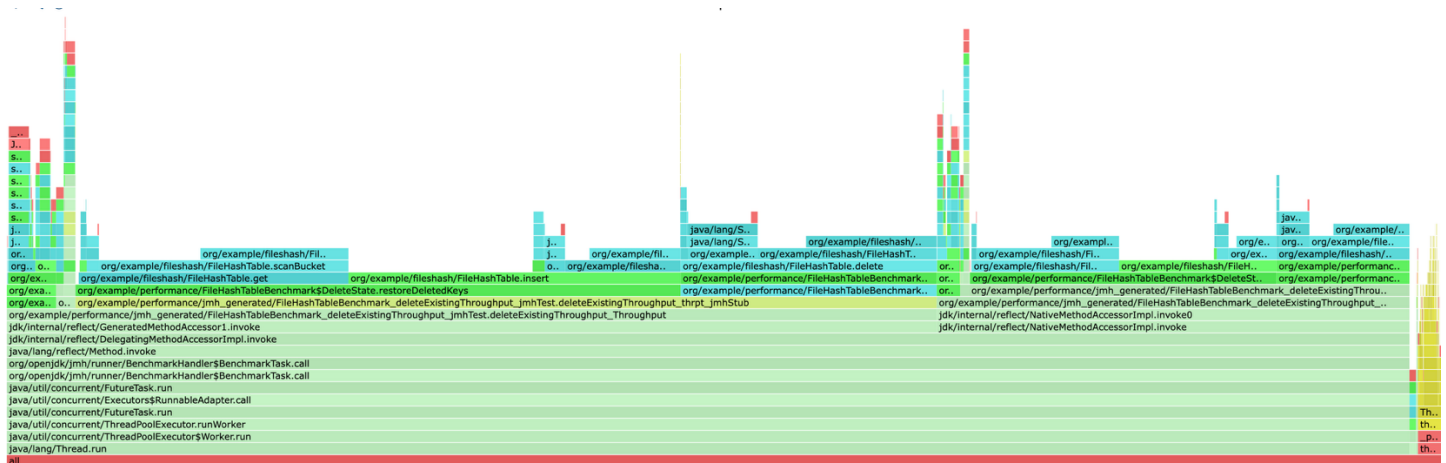


Рисунок 15 – CPU-профиль для операции delete

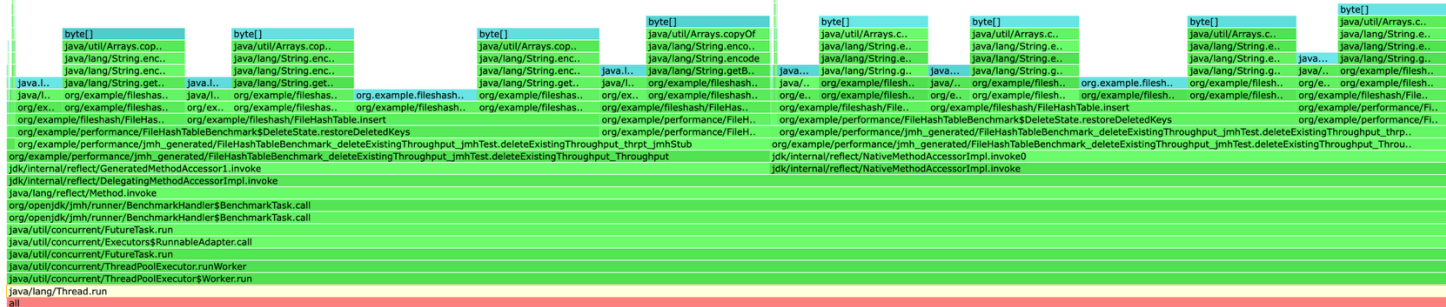


Рисунок 16 – Профиль аллокаций для операции delete

Latency операции delete в целом растет с увеличением N. Основное время уходит на поиск элемента через scanBucket и сравнение ключей, что видно в CPU profile. Allocation profile показывает аллокации string и byte при работе с данными. Throughput постепенно снижается по тем же причинам: каждая операция становится немного дороже. При этом поведение в целом более стабильное, чем у insert, так как delete не вызывает split бакетов и не приводит к тяжелым операциям перераспределения данных.

Таблица 3 – Результаты latency бенчмарка PerfectHash(среднее по итерациям)

N	Build, нс/оп	Lookup hit, нс/оп
20000	2,083,224 ± 371,357	32.461 ± 0.554
40000	3,873,799 ± 196,987	33.374 ± 0.61
60000	6,698,225 ± 565,696	34.028 ± 0.796
80000	8,394,357 ± 766,261	34.055 ± 0.479
100000	11,968,082 ± 604,172	34.262 ± 0.483
120000	13,754,710 ± 687,352	34.776 ± 0.634

Таблица 4 – Результаты бенчмарка throughput PerfectHash(среднее по итерациям)

N	Lookup hit, ops/s
20000	30,565,589 ± 506,364
40000	29,324,990 ± 643,708
60000	29,385,937 ± 606,393
80000	29,092,536 ± 517,075
100000	28,767,534 ± 584,676
120000	28,906,611 ± 610,888

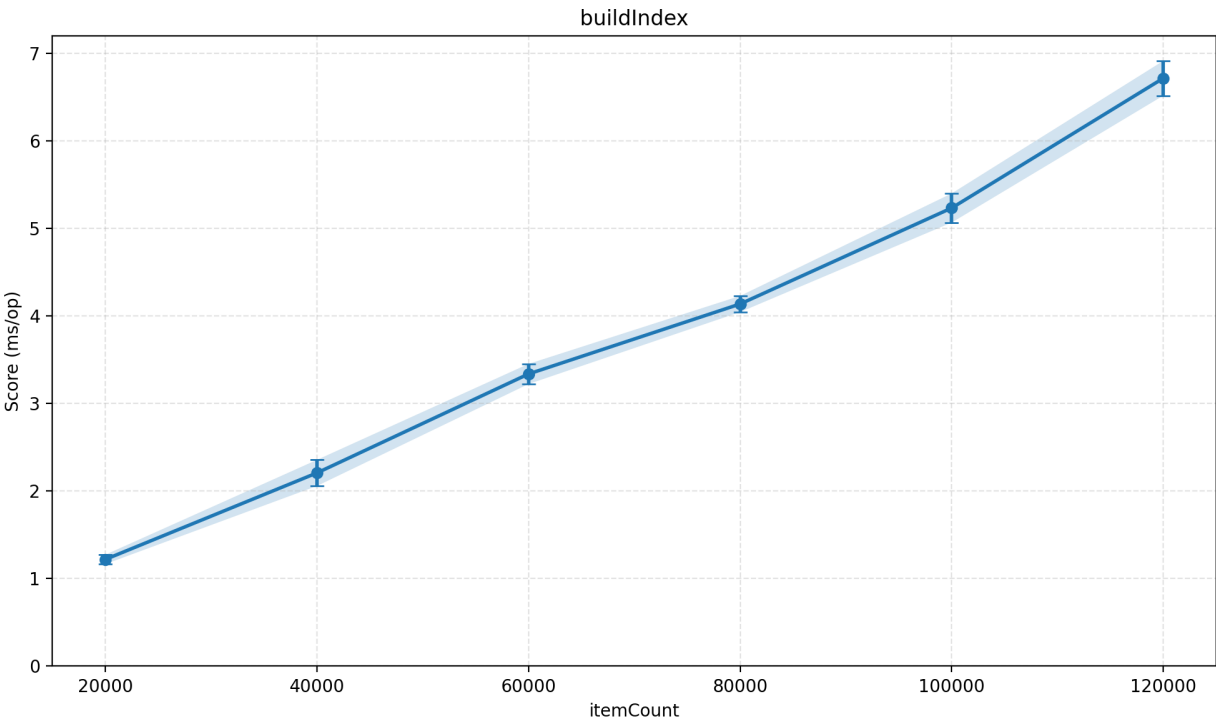


Рисунок 17 – График latency для построения PerfectHash

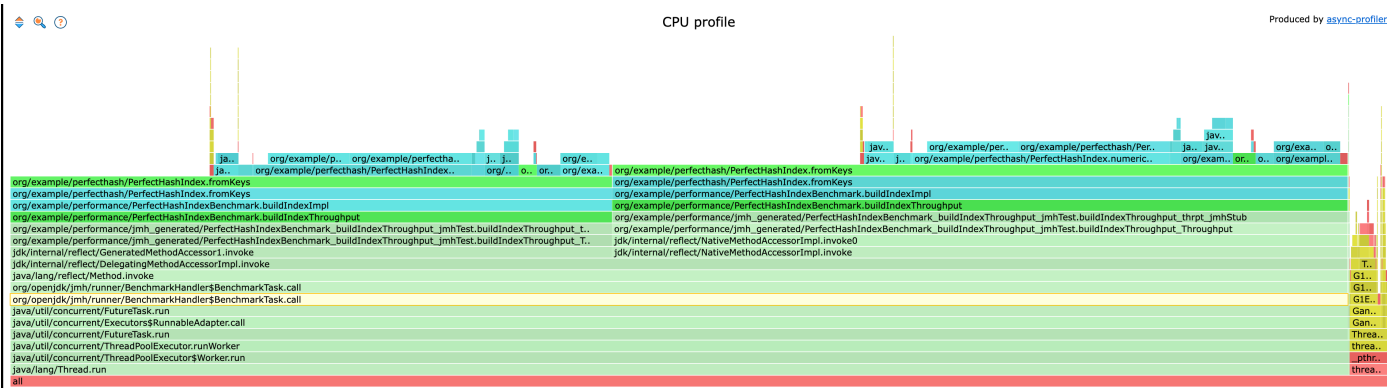


Рисунок 18 – CPU-профиль для операции build

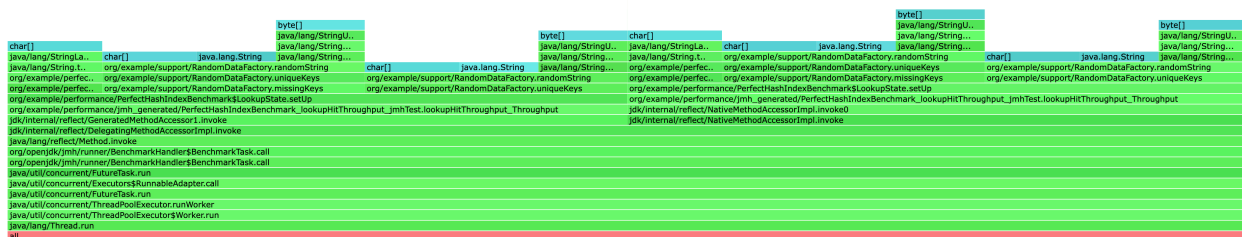


Рисунок 22 – Профиль аллокаций для операции lookup

Latency lookup’a остается практически постоянной и слабо зависит от размера данных, что соответствует ожидаемой сложности $O(1)$. Небольшой рост с увеличением N связан с ухудшением кэш-локальности и увеличением объема данных, но сам поиск выполняется без сканирования благодаря отсутствию коллизий. Профилирование показывает, что основное время уходит на вычисление хеша и сравнение строки, без дополнительных затрат на обход структур.

LSH

Таблица 5 — throughput построение / добавление / LSH-поиск

N	построение, нс/оп	добавление, нс/оп	LSH-поиск, нс/оп	Full scan, нс/оп
2000	153,474 ± 9,232	16.97 ± 0.289	583,390 ± 15,945	2,850,617 ± 83,823
4000	288,424 ± 24,184	21.715 ± 0.256	2,657,842 ± 47,052	12,327,253 ± 118,776
6000	421,192 ± 194,459	16.538 ± 0.721	4,531,763 ± 82,205	28,651,632 ± 258,473
8000	438,319 ± 213,788	17.256 ± 0.107	21,001,447 ± 154,770	49,653,724 ± 775,453
10000	397,161 ± 67,577	18.338 ± 0.181	18,036,467 ± 375,963	76,282,197 ± 1,494,091
12000	453,650 ± 76,085	18.697 ± 0.244	22,276,540 ± 694,592	109,609,569 ± 1,944,071

Таблица 6 —throughput построение / добавление / LSH-поиск

N	Build, ops/s	Add, ops/s	LSH search, ops/s	Full scan, ops/s
2000	20,178 ± 186	58,417,780 ± 450,006	1,783.265 ± 2.166	338.941 ± 2.38
4000	15,015 ± 221	45,756,728 ± 627,177	390.413 ± 5.21	81.898 ± 1.161
6000	8,107 ± 243	59,768,996 ± 2,673,924	225.807 ± 2.955	36.272 ± 0.492
8000	5,976 ± 114	58,239,591 ± 803,456	48.535 ± 0.761	20.232 ± 0.368

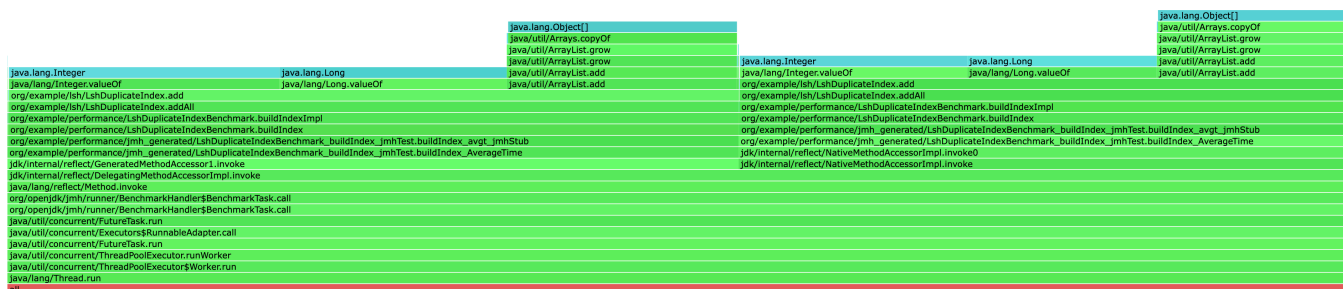


Рисунок 25 – Профиль аллокаций для операции build

Построение LSH-индекса демонстрирует близкое к линейному росту времени с увеличением числа точек. Это ожидаемо, так как для каждой точки вычисляется хэш и происходит добавление в структуру. Небольшие колебания на графике связаны с внутренними реаллокациями ArrayList и HashMap, что подтверждается профилированием.

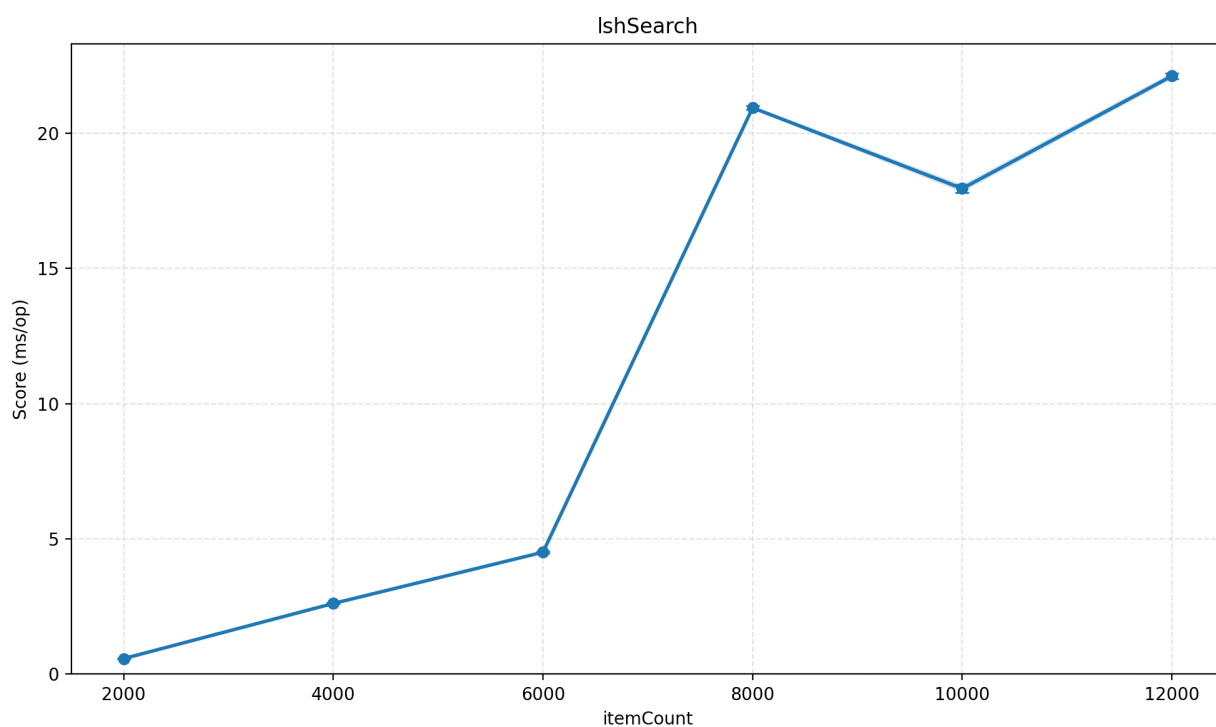
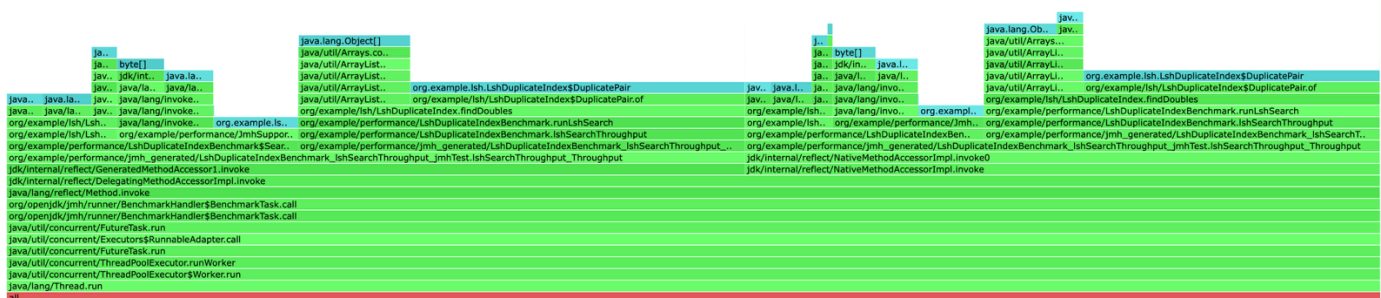
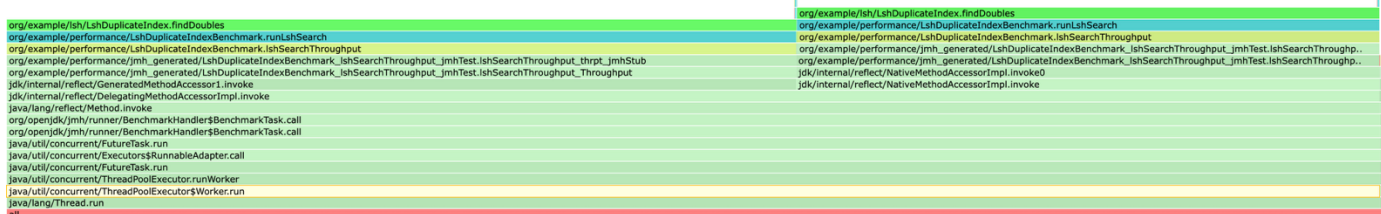


Рисунок 26 –График latency для операции search



Latency поиска в LSH в целом небольшая, но наблюдается резкий скачок примерно на $N \approx 8000$. Это связано с тем, что распределение точек по бакетам становится менее равномерным, и в отдельных бакетах накапливается больше элементов. В результате внутри таких бакетов возрастает число попарных сравнений, что резко увеличивает время выполнения. Профилирование показывает, что основное время уходит именно на перебор пар в `findDoubles`, поэтому поведение определяется не самим хэшированием, а размером отдельных бакетов.

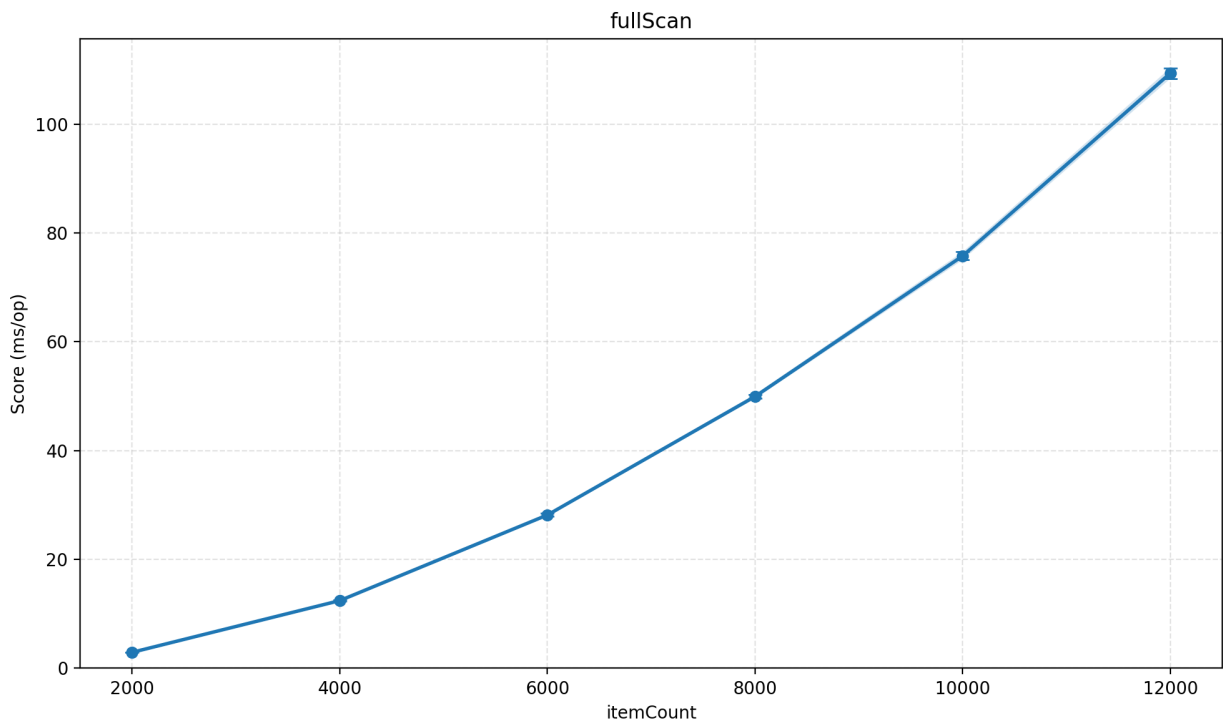


Рисунок 29 – График latency для fullScan

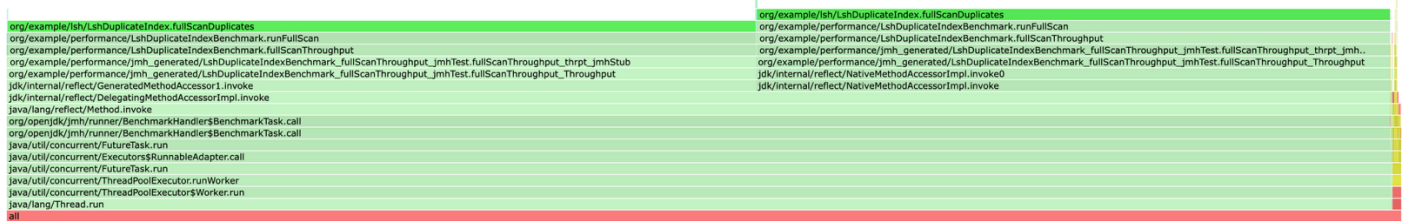


Рисунок 30 – CPU-профиль для fullScan

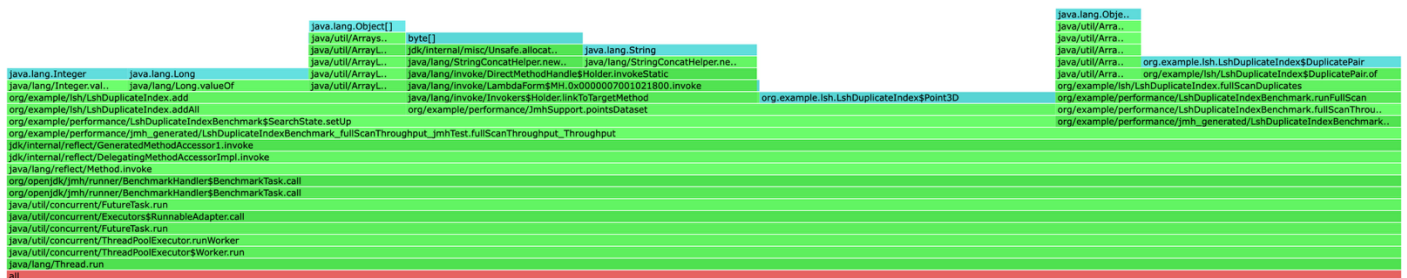


Рисунок 31 – Профиль аллокаций для fullScan

Полный перебор показывает ожидаемый квадратичный рост времени. Каждая точка сравнивается со всеми остальными, поэтому с увеличением N резко растет число сравнений и латентность. Профилирование подтверждает, что основное время уходит именно на двойной цикл сравнения точек.

Выводы

Наиболее стабильные результаты по времени показывает PerfectHashIndex: операции lookup практически не зависят от размера данных и остаются близкими к константным. Основные затраты пришлись на этап построения и вычисление numericKey, тогда как сам поиск выполняется быстро и без дополнительных обходов.

У FileHashTable операции get, update и delete демонстрируют ровное поведение, а алгоритмическая $O(1)$ для get была подтверждена экспериментально — на бенчмарке с фиксированным hot-set из 64 ключей latency остаётся плоской на уровне ~ 67 нс независимо от N . Наблюдаемый рост на сценарии случайных запросов по всему диапазону объясняется не алгоритмом, а cache miss penalty: с увеличением N работающий набор bucket-файлов перестаёт помещаться в L1/L2. Основная стоимость lookup связана с линейным сканированием бакета и сравнением ключей. Операция insert менее стабильна и имеет немонотонную форму с пиками при $N \approx 4000$ и $N \approx 8000$ — это бимодальная природа вставки в extendible hashing: обычная вставка ~ 1 мкс, split bucket ~ 200 мкс, split с doubleDirectory ~ 1 мс. Доля каждого типа в батче зависит от текущего globalDepth, что и даёт характерные пики на границах его удвоений.

В случае LSH поиск через индекс существенно быстрее полного перебора, так как количество сравнений сокращается за счёт предварительного разбиения на бакеты. Однако время выполнения зависит от распределения точек: при неравномерном распределении увеличивается число кандидатов внутри бакетов, что приводит к росту времени в findDoubles. Базовый метод fullScanDuplicates демонстрирует ожидаемое квадратичное поведение.

С точки зрения узких мест:

- для FileHashTable — scanBucket и matchesFixedBytes на чтении, а также splitBucket, rewriteBucket, readEntries и doubleDirectory при вставке;
- для PerfectHashIndex — этап построения, в первую очередь numericKey и подбор хэш-функций;
- для LSH — перебор кандидатов в findDoubles и полный перебор в baseline.

Возможные улучшения:

для FileHashTable — убрать промежуточные byte[] в keyBytes/readFixedBytes за счёт прямого сравнения по MappedByteBuffer, увеличить bucketCapacity для уменьшения частоты split-событий; для PerfectHashIndex улучшения уже внесены — устранены аллокации HashMap/ArrayList при построении, что дало $\sim 1.8\times$ ускорение build; для LSH — оптимизация параметров хеширования и фильтрация кандидатов перед полным сравнением.